Sakto: System Architecture

Barangay tanod incentive documents for the Province of Iloilo (Provincial Ordinance 2020-240)

Version 15 October 2026 **Stage** Pilot build **For** OPPDC, the investor and the Sakto team

1. What Sakto does

Each year, every barangay in Iloilo lists its tanods for the provincial incentive. The lists go to the municipality, where the Liga Rep checks them and prepares the masterlist, the payroll, a cover sheet and each barangay's certification. The provincial office (OPPDC) then pays from the payroll. Sakto replaces the hand-encoded spreadsheets with one shared list. Barangay secretaries encode or scan their tanods, Sakto checks every rule as they go, and the documents print already filled in and consistent.

Who uses it

Role	What they do in Sakto	Sees
Barangay secretary	Encodes their barangay's tanods by typing, scanning IDs or scanning a written list. Ticks requirements and prints the Barangay Certification. Works offline.	Own barangay; other barangays' progress (read-only)
Liga Rep	Approves secretaries, sets the barangay deadline, reviews and fixes every list, locks and submits the year, prints the masterlist, payroll, cover sheet and certifications.	Whole municipality
Provincial office (OPPDC)	Oversees all 43 municipalities and opens any one read-only. Sets the provincial deadline, changes tanod limits when an ordinance or amendment says so, manages accounts and answers problem reports.	Whole province
Sakto team	The same administrator screens as OPPDC, for support.	Whole province

2. System overview

Sakto is a web app installed from the browser (a *progressive web app*). There is no custom server: the app talks directly to **Supabase**, a hosted Postgres database with sign-in, file storage and serverless functions. Every access rule is enforced inside the database, so the app cannot show or change data a person is not allowed to touch.

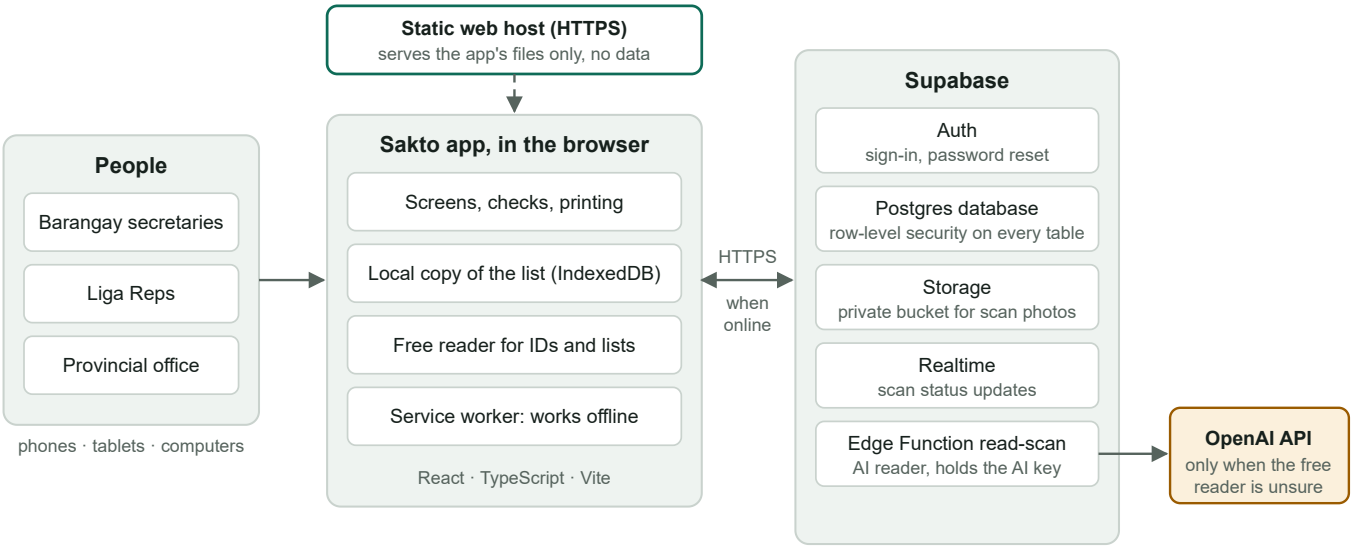


Figure 1. The parts of Sakto and how they connect. The browser holds a full working copy of the list, so encoding continues without signal.

> **Design choices in one line each.** Offline first, because barangays often have weak signal. The rules live in the database, so security doesn't depend on the app. No custom server, so there is little to run or maintain. Documents are generated from one list, so the masterlist, payroll and certifications can't disagree.

3. The app

Technology

- **React 19** and **TypeScript**, built with **Vite**
- **zustand** for state, saved to the device in **IndexedDB**
- **vite-plugin-pwa**: installable, cached for offline use
- **tesseract.js**: the free on-device reader for IDs and typed lists
- **qrcode**: the cover sheet's verification code
- **@supabase/supabase-js**: sign-in, data, photos

Main screens

- **Dashboard**: progress, deadlines and what needs attention. Administrators also get *All of Iloilo*.
- **Setup** (Liga Rep): year, amount, signatories, deadlines
- **Barangays / Tanods**: the list, requirements, scanning, tanod limit
- **Review**: every blocking issue, the ObR check, locking the year
- **Print** and **Templates**: the documents, filled in or blank
- **Records**, **Accounts** and **Help** (with problem reports)

Documents

The documents are generated in the browser from the list and printed on long bond paper (8.5 × 13 in): the **Masterlist**, the **Provincial Payroll** (landscape, with signature boxes A–E), the **Submission cover sheet** and the **Barangay Certification**. The cover sheet carries a check code and a QR code built from the award numbers, names, year and amount, so the province can confirm that a printed set matches what was encoded. While any blocking issue remains, every page prints with a *DRAFT* watermark.

The checks behind the watermark are:

- a barangay is over its limit;
- a name is missing;

- a person is listed twice;
- an ID photocopy lacks its 3 signatures;
- the ObR payee or amount doesn't match.

4. Working offline: how the sync works

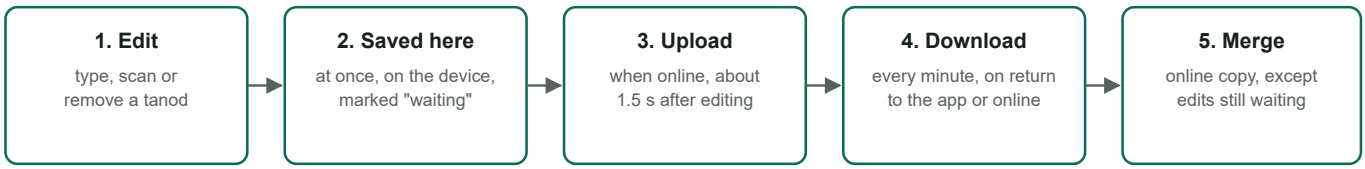


Figure 2. Every change is saved on the device first. Uploading and downloading happen in the background and never run at the same time.

- **What each device downloads.**
  - A secretary: only their barangay.
  - A Liga Rep: the whole municipality.
  - An administrator: the municipality they opened, read-only. An administrator's device never uploads.
- **Conflicts.** The last change to reach Supabase wins. A row edited on this device and not yet uploaded is kept, never overwritten by the download. Row order is saved separately (`set_roster_order`), so reordering on two devices doesn't duplicate anyone.
- **Locked lists.** Once the Liga Rep locks the year, the database refuses secretaries' changes. The app says why and offers to discard unsent changes.
- **Always visible.** A badge on every screen says whether the work has reached Supabase ("Saved on this device · 3 waiting for internet", "All changes saved online").
- **What needs internet.** The first sign-in on a device, uploading scan photos, the AI reader, tanod limit changes and problem reports. Encoding, reviewing and printing work offline.

5. Scanning: from photo to list

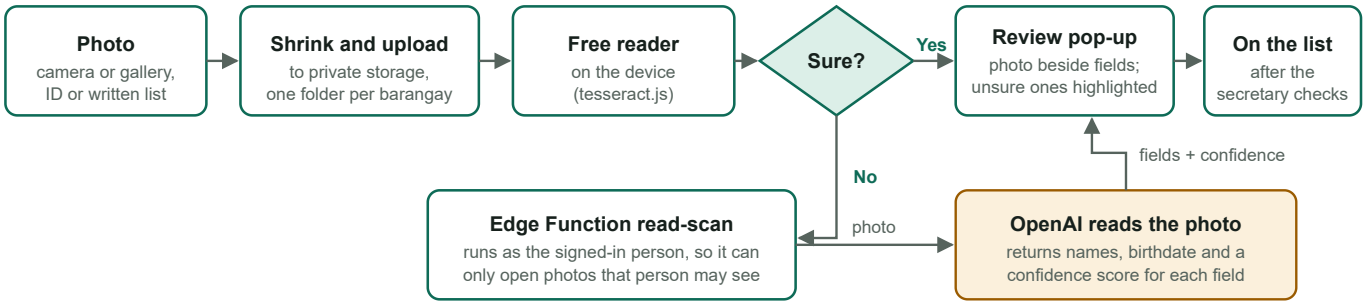


Figure 3. The free reader handles clear, typed IDs and lists on its own. The AI reader is only asked when the free reader isn't sure, which is always the case for handwriting.

- **Scan types.**
  - An *ID* (front, and the back if needed) becomes one tanod.
  - A *list* (a typed or handwritten page) becomes one row per person, shown as a table to tick and correct. Names already listed, and rows past the limit, start unticked.
  - A *document* photo (certification, designation, SPA) is attached as proof.
- **AI provider.** OpenAI by default (Responses API with a strict JSON schema; model set by `OPENAI_MODEL`). Anthropic Claude is supported as an alternative (`AI_PROVIDER`). The API key lives only in the Edge Function's

secrets, never in the app or on devices.

- **Photo lifetime.** Photos stay until the year is closed, then they are deleted permanently.

6. Data model

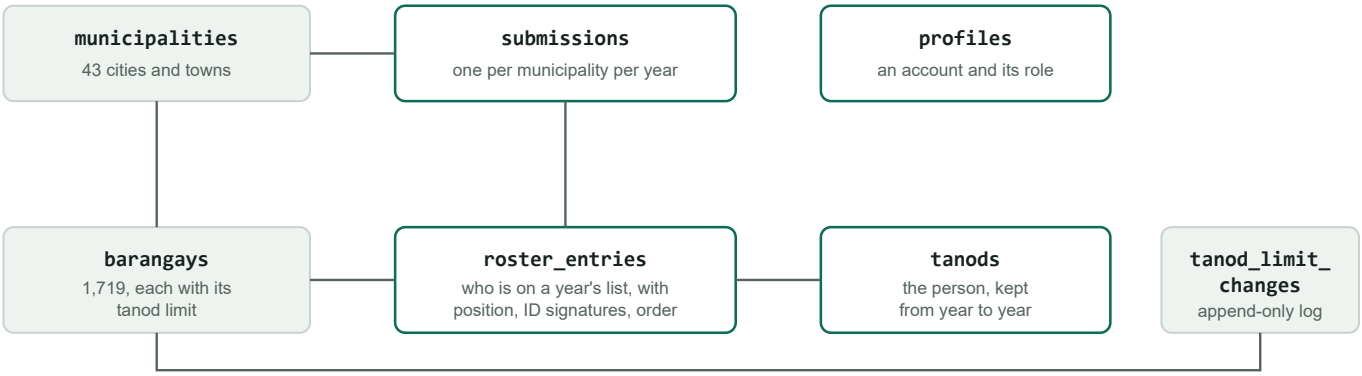


Figure 4. The core tables. Each year's list is a `submission`. A tanod who serves again next year is the same `tanods` row, so their history builds up over the years.

Table	Holds
`municipalities`, `barangays`	The provincial reference list. Only administrators can change it. Every limit change goes through `set_tanod_limit` and is logged.
`profiles`	One per account: name, phone, role (pending, secretary, rep, admin), municipality and barangay. New sign-ups start as *pending*.
`submissions`	A municipality's year: status, amount per tanod, signatories, ObR details, barangay deadline.
`tanods`, `roster_entries`	The people, and who is on which year's list in which barangay, in order.
`barangay_requirements`	Per barangay per year: certification, proof of designation, SPA, focal person and phone.
`tanod_departures`	Why someone on last year's list left: resigned, removed, deceased, moved, other. Powers the Records page.
`scans`	Each scanned ID, list or document: kind, status (uploading, reading, ready, failed, saved), what was read, photo locations.
`year_deadlines`	The provincial deadline for municipalities, per year.
`tanod_limit_changes`	Old and new limit, the year, the reason (ordinance or resolution), who and when. Nobody can edit or delete it.
`support_requests`	Problem reports from the Help page, with technical details, for the provincial office.

Database functions: `set_roster_order` (saves list order), `municipality_progress` (other barangays' counts for secretaries) and `set_tanod_limit` (administrators only).

7. The year, from start to close

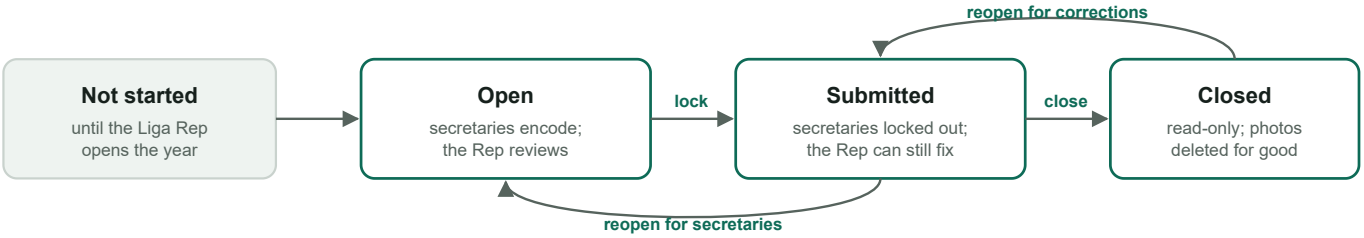


Figure 5. Each municipality's year moves through four states. Only the Liga Rep moves it, and the database enforces what each state allows.

Deadlines run alongside: the provincial office sets one date for all municipalities, and each Liga Rep sets one for their barangays. A banner counts down on every screen and turns red 7 days before. Next year, the Rep opens a new year and secretaries can copy last year's tanods in one step, recording why anyone who left is gone.

8. Security and privacy

Who can do what

Action	Secretary	Liga Rep	Provincial office
See their own barangay's list	**Yes**	**Yes**	**Yes**
Change their own barangay's list	**While open**	**Until closed**	No (read-only)
See and change every barangay in the municipality	Progress only	**Yes**	**See only**
Lock, reopen and close the year	No	**Yes**	No
Set the barangay deadline / the provincial deadline	No	**Barangay**	**Provincial**
Approve accounts	No	**Secretaries**	**Everyone**
Change a barangay's tanod limit (logged)	No	No	**Yes**
See every municipality, problem reports	No	No	**Yes**

How it is enforced

- **Row-level security on every table.** Shared helper functions decide access from the signed-in person's profile: `can_access_barangay`, `can_edit_roster`, `can_manage_municipality`, `can_read_municipality`, `my_role`. A request the rules don't allow returns nothing or is refused, whatever the app sends.
- **Changes that touch several rows** (list order, tanod limits) go through database functions that check the role first.
- **Accounts.**
  - Self sign-up creates a *pending* account with no access until approved.
  - Nobody can raise their own role; a database trigger blocks it.
  - The two built-in administrator accounts (OPPDC-ADMIN, DEV-Admin) sign in with a username and must choose a new password at their first sign-in.
  - Only bcrypt hashes of their starting passwords are stored in the setup script.
- **Keys.**
  - The app holds only Supabase's publishable key, which is safe to expose because the security rules do the protecting.

    - The service-role key is never in the app.
    - The AI key exists only in the Edge Function's secrets.
- **Data in transit and at rest.** All traffic uses HTTPS. Scan photos sit in a private storage bucket, readable only by people who can see that barangay, and are deleted when the year closes. Each device keeps a copy of its own list for offline work; signing out warns if anything hasn't been uploaded.

> **Data privacy (RA 10173).** Sakto holds personal data: names, and ID photos while a year is open. When the AI reader is used, a photo is sent to OpenAI to be read. Before the pilot:
> - the privacy notice must name OpenAI;
> - tanods must give consent for their ID photos;
> - demo material must use team members' own IDs, with their consent.

9. Deployment and operations

Part	How it is set up
Database	Run the scripts in `supabase/migrations` in order in the Supabase SQL Editor, 0000 to 1000 (eleven scripts). Then run `supabase/seed/admin_accounts.sql` once, to create the two administrator accounts.
AI reader	Deploy the Edge Function `read-scan` with JWT verification on. Add the secret `OPENAI_API_KEY`; `OPENAI_MODEL` and `AI_PROVIDER` are optional. Set a monthly spending limit on the OpenAI account.
App	Set `VITE_SUPABASE_URL` and `VITE_SUPABASE_PUBLISHABLE_KEY` (optional: `VITE_SUPPORT_PHONE`, `VITE_SUPPORT_EMAIL`). Build with `npm run build`, then put the static files on any HTTPS host. HTTPS is required for the camera, installing and offline use.
Updates	A new build is picked up the next time the app is opened. People are asked before it switches, so no one loses an edit.
Backups	Supabase's own database backups (depending on plan), plus a backup file anyone can save from Setup.
Hosting choice	Supabase's cloud by default. If policy requires the data on a provincial server, Supabase can be self-hosted at the Capitol; only the app's server address changes.

10. Known limits and next steps

- The AI reader has been built and type-checked but not yet tried on real IDs and handwritten lists. Model choice and cost should be measured during the pilot.
- Username accounts have no mailbox, so a forgotten password is reset by the Sakto team in the SQL Editor; the setup script has the command.
- Some things need internet: the first sign-in on a device, photo uploads, limit changes and the AI reader. A file hand-off for barangays with no signal is planned.
- Hosting (the HTTPS address) is still to be chosen.
- A dedicated mobile scanning app (Expo) is planned. It would use the same Supabase back end and rules.